

ANEXO B

Manual de instalación, configuración y administración

Checkpoint — Entorno local y producción en AWS

ÍNDICE

1. Introducción	3
2. Requisitos previos	3
2.1 Hardware	3
2.2 Software necesario	3
3. Instalación	3
3.1 Clonar el repositorio	3
3.2 Instalar dependencias del frontend	3
3.3 Instalar dependencias del backend	4
4. Configuración	4
4.1 Perfiles de configuración del backend	4
4.2 Configuración de la base de datos local	4
4.3 Configuración del frontend	4
4.4 CORS	4
4.5 Seguridad	5
5. Arranque en entorno local	5
5.1 Levantar MySQL con Docker	5
5.2 Arrancar el backend	5
5.3 Arrancar el frontend	5
6. Despliegue en producción (AWS)	6
6.1 Backend en EC2	6
6.2 Base de datos en RDS	6
6.3 Frontend en S3	6
6.4 Verificar el despliegue	7
7. Administración y mantenimiento	7
7.1 Gestión de datos	7
7.2 Actualización de la aplicación	7
7.3 Copias de seguridad	7
8. Solución de problemas	8
9. Desinstalación (entorno local)	9
10. Recursos adicionales	9

1. Introducción

Este manual está dirigido a la persona responsable de instalar, configurar y mantener Checkpoint. Cubre tanto el entorno de desarrollo local como el despliegue en producción sobre AWS.

Checkpoint es una aplicación web con arquitectura cliente-servidor en tres capas: un frontend Angular, un backend Spring Boot y una base de datos MySQL. En local se levanta con Docker; en producción usa Amazon EC2, RDS y S3.

2. Requisitos previos

2.1 Hardware

No hay requisitos de hardware específicos. Cualquier máquina de desarrollo habitual es suficiente para el entorno local. Para el despliegue en AWS, los recursos mínimos vienen determinados por el tipo de instancia EC2 y el plan de RDS elegidos.

2.2 Software necesario

Para ejecutar Checkpoint en local hacen falta las siguientes herramientas:

- Java 17.
- Maven (incluido en el proyecto o instalado globalmente).
- Node.js (versión LTS) y npm.
- Angular CLI - instalar con: `npm install -g @angular/cli`
- Docker (para levantar MySQL en contenedor).
- Git.

Las versiones de las dependencias del proyecto están fijadas en `pom.xml` (backend) y `package.json` (frontend). Usar versiones distintas a las especificadas puede causar incompatibilidades, especialmente con Lombok y la versión de Java.

3. Instalación

3.1 Clonar el repositorio

El código fuente está en GitHub:

```
git clone https://github.com/AlbaBer15/Checkpoint-proyecto-Angular.git
cd Checkpoint-proyecto-Angular
```

El repositorio contiene tanto el proyecto Angular (frontend) como el proyecto Spring Boot (backend) en la misma raíz.

3.2 Instalar dependencias del frontend

Desde la carpeta del proyecto Angular:

```
npm install
```

3.3 Instalar dependencias del backend

Maven descarga las dependencias automáticamente al arrancar o compilar. No hay ningún paso manual adicional. Si se quieren descargar antes de arrancar:

```
mvn dependency:resolve
```

4. Configuración

4.1 Perfiles de configuración del backend

El backend usa tres archivos de propiedades separados:

- `application.properties` — configuración base y perfil de desarrollo local (MySQL en `localhost:3306`, usuario `root`, contraseña `root`).
- `application-prod.properties` — configuración de producción (URL de RDS, usuario `admin`, contraseña).
- `application-test.properties` — configuración para los tests de integración (base de datos H2 en memoria, `ddl-auto=create-drop`).

El perfil activo se selecciona en el arranque. Por defecto (sin indicar perfil) se usa la configuración base, que apunta al entorno local.

4.2 Configuración de la base de datos local

El archivo `application.properties` ya está en el repositorio con los valores de desarrollo preconfigurados. No hace falta modificar nada si se levanta MySQL con el comando Docker del apartado 5.1.

Nota: Si ya se tiene MySQL instalado localmente en lugar de Docker, asegurarse de que existe una base de datos llamada 'checkpoint' y de que las credenciales coinciden con las del archivo `application.properties`.

4.3 Configuración del frontend

El frontend usa dos archivos de entorno:

- `src/environments/environment.ts` — apunta a `http://localhost:8080/api` (desarrollo).
- `src/environments/environment.prod.ts` — apunta a `http://3.221.75.194:8080/api` (producción).

Angular selecciona el archivo correcto automáticamente según el comando de compilación (`ng serve` usa `development`, `ng build --configuration production` usa `production`).

4.4 CORS

El backend tiene CORS configurado para aceptar peticiones de tres orígenes:

- `http://localhost:4200` (desarrollo local).
- `http://3.221.75.194:8080` (Swagger UI en producción).
- `http://checkpoint-frontend.s3-website-us-east-1.amazonaws.com` (frontend en S3).

Si se despliega el frontend en una URL diferente, hay que añadirla a la configuración de CORS en `SecurityConfig.java` y recompilar el backend.

4.5 Seguridad

La aplicación no tiene sistema de autenticación. Todas las rutas del API (/api/**) son públicas. CSRF está deshabilitado porque la aplicación funciona como API REST sin sesiones ni cookies.

Las credenciales de la base de datos de producción están hardcodeadas en `application-prod.properties`. Esto es una limitación conocida del proyecto: en un entorno real deberían gestionarse como variables de entorno o mediante un servicio de secretos.

5. Arranque en entorno local

5.1 Levantar MySQL con Docker

Ejecutar el siguiente comando para crear y arrancar el contenedor:

```
docker run --name checkpoint-db \
-e MYSQL_ROOT_PASSWORD=root \
-e MYSQL_DATABASE=checkpoint \
-p 3306:3306 \
-d mysql:8
```

Si el contenedor ya existe de una ejecución anterior:

```
docker start checkpoint-db
```

Nota: Comprobar que el puerto 3306 no está ocupado por otro proceso antes de ejecutar el contenedor. Si lo está, detener el proceso que lo usa o cambiar el puerto de mapeo (y actualizar `application.properties` en consecuencia).

5.2 Arrancar el backend

Desde la carpeta raíz del proyecto backend (donde está `pom.xml`):

```
mvn spring-boot:run
```

El servidor arranca en el puerto 8080. La primera vez, Spring crea las tablas en la base de datos e inserta los datos iniciales (6 categorías y 8 logros) automáticamente mediante `DataInitializer`. Si la tabla ya tiene datos, `DataInitializer` no vuelve a insertarlos.

Para verificar que el backend responde:

```
http://localhost:8080/swagger-ui/index.html
```

5.3 Arrancar el frontend

Desde la carpeta del proyecto Angular:

```
ng serve
```

La aplicación queda disponible en:

```
http://localhost:4200
```

6. Despliegue en producción (AWS)

La infraestructura de producción usa tres servicios de AWS:

- Backend: instancia EC2 (IP pública: 3.221.75.194, puerto 8080).
- Base de datos: Amazon RDS con MySQL 8 (checkpoint-db.cud820283dfn.us-east-1.rds.amazonaws.com).
- Frontend: bucket S3 configurado como sitio web estático.

6.1 Backend en EC2

Para compilar el JAR ejecutable desde la carpeta del proyecto backend:

```
mvn package -DskipTests
```

El JAR se genera en la carpeta target/. Transferirlo a la instancia EC2:

```
scp -i labsuser.pem target/backend-0.0.1-SNAPSHOT.jar ec2-user@3.221.75.194:~
```

Para conectarse a la instancia, acceder a la consola de AWS -> EC2 -> seleccionar la instancia -> Connect -> EC2 Instance Connect -> Connect. Se abre un terminal en el navegador desde el que ejecutar los comandos siguientes.

Una vez en EC2, arrancarlo con nohup para que siga activo al cerrar la terminal:

```
nohup java -jar backend-0.0.1-SNAPSHOT.jar --spring.profiles.active=prod &
```

6.2 Base de datos en RDS

La URL de conexión configurada en application-prod.properties es:

```
checkpoint-db.cud820283dfn.us-east-1.rds.amazonaws.com
```

El esquema se crea automáticamente al arrancar el backend (ddl-auto=update). Los datos iniciales los inserta DataInitializer la primera vez que las tablas están vacías.

En el grupo de seguridad asociado a la instancia RDS hay que añadir una regla de entrada que permita tráfico TCP en el puerto 3306 desde el grupo de seguridad de la instancia EC2.

6.3 Frontend en S3

Generar el build de producción:

```
ng build --configuration production
```

El build genera la carpeta dist/checkpoint/browser/ con los archivos estáticos compilados.

Subir el contenido de la carpeta dist/checkpoint/browser/ al bucket checkpoint-frontend. El bucket debe tener habilitado el alojamiento de sitio web estático con index.html configurado como documento principal y como documento de error. Configurar index.html también como documento de error es necesario para que el enrutado de Angular funcione al acceder directamente a una URL que no sea la raíz.

6.4 Verificar el despliegue

Una vez arrancado el backend y subido el frontend, las URLs de acceso son:

- Aplicación: <http://checkpoint-frontend.s3-website-us-east-1.amazonaws.com>
- API: <http://3.221.75.194:8080/api>
- Swagger UI: <http://3.221.75.194:8080/swagger-ui/index.html>

7. Administración y mantenimiento

7.1 Gestión de datos

No existe panel de administración. El acceso directo a los datos se hace a través de un cliente MySQL conectado a la base de datos correspondiente (localhost:3306 en local, URL de RDS en producción).

La API REST también permite operar sobre los datos directamente. La documentación interactiva de todos los endpoints está disponible en Swagger UI, tanto en local como en producción.

7.2 Actualización de la aplicación

Para actualizar el backend en local, detener el proceso con Ctrl+C y volver a ejecutar `mvn spring-boot:run`. Si se han añadido columnas o tablas nuevas, `ddl-auto=update` las aplica automáticamente al arrancar. Para el frontend, detener `ng serve`, hacer los cambios y volver a arrancarlo.

En producción, compilar el nuevo JAR con `mvn package`, transferirlo a EC2 y reiniciar el proceso. Para el frontend, generar el nuevo build con `ng build --configuration production` y subir los archivos a S3.

Nota: `ddl-auto=update` aplica cambios estructurales incrementales pero no elimina columnas ni tablas existentes. Para operaciones destructivas hay que ejecutar el SQL manualmente.

7.3 Copias de seguridad

No hay ningún proceso de backup automatizado definido en el proyecto. En un entorno de producción real habría que configurar snapshots automáticos en RDS o exportaciones periódicas con `mysqldump`. Para el entorno local, la persistencia depende del contenedor Docker: si se elimina el contenedor con `docker rm`, los datos se pierden.

Nota: Para conservar los datos del contenedor local entre reinicios, el contenedor creado con el comando de 5.1 persiste mientras no se elimine con `docker rm`. Detenerlo con `docker stop checkpoint-db` no borra los datos.

8. Solución de problemas

El backend no arranca: error de conexión a la base de datos

La causa más habitual es que el contenedor Docker no está en ejecución. Comprobarlo con:

```
docker ps
```

Si checkpoint-db no aparece en la lista, arrancarlo con `docker start checkpoint-db`. Si tampoco existe, crearlo con el comando del apartado 5.1.

El backend no arranca: puerto 8080 ocupado

Hay otro proceso usando el puerto 8080. En Linux y Mac:

```
lsof -i :8080
```

En Windows:

```
netstat -ano | findstr :8080
```

Detener el proceso que ocupa el puerto o cambiar el puerto del backend en `application.properties` (`server.port`).

El frontend no arranca: puerto 4200 ocupado

Usar un puerto alternativo:

```
ng serve --port 4201
```

Si se cambia el puerto, actualizar también la configuración de CORS en el backend para incluir la nueva URL.

Incompatibilidades de versiones (Lombok, Maven, Java)

Si Maven falla al compilar con errores relacionados con Lombok o con la versión de Java, comprobar que la JDK activa es la 17 con:

```
java -version
```

Las versiones exactas de todas las dependencias están fijadas en `pom.xml`. No modificarlas sin verificar la compatibilidad entre ellas.

Los logs del backend

Spring Boot escribe los logs en la consola con nivel INFO por defecto. El único log de error explícito está en `GlobalExceptionHandler`, que registra las excepciones no controladas (errores 500). No hay archivo de log ni configuración de niveles personalizada. Para ver los logs en tiempo real basta con tener la consola del proceso visible.

Errores 404 o 400 en la API

Consultar Swagger UI para verificar la forma exacta de la petición (ruta, método HTTP y cuerpo esperado). Swagger está disponible en local en `http://localhost:8080/swagger-ui/index.html` y en producción en `http://3.221.75.194:8080/swagger-ui/index.html`.

9. Desinstalación (entorno local)

Para eliminar Checkpoint del entorno local:

- Detener el contenedor Docker: `docker stop checkpoint-db`
- Eliminar el contenedor y sus datos: `docker rm checkpoint-db`
- Eliminar la imagen de MySQL si ya no se necesita: `docker rmi mysql:8`
- Detener el proceso del backend (Ctrl+C en la consola donde esté ejecutándose).
- Detener el servidor de desarrollo del frontend (Ctrl+C en la consola donde esté ejecutándose).
- Eliminar la carpeta del repositorio clonado.

Los datos de la base de datos se eliminan al borrar el contenedor Docker. No hay archivos adicionales fuera de la carpeta del repositorio.

10. Recursos adicionales

Repositorio del proyecto en GitHub:

<https://github.com/AlbaBer15/Checkpoint-proyecto-Angular>

Swagger UI en local (documentación interactiva de la API):

<http://localhost:8080/swagger-ui/index.html>

Swagger UI en producción:

<http://3.221.75.194:8080/swagger-ui/index.html>

Para incidencias o consultas relacionadas con el proyecto, puede abrirse un issue directamente en el repositorio de GitHub.